

Homework 3 question 3 report

Question 3 is an implementation of DQN for the cartpole. Initial test code was provided, and I built DQN on that.

Code implementation:

- Added a class `replay_buffer` where we store the experience of the DQN
- Implemented epsilon greedy policy where best action is chosen by examining the max q value
- Added class `train_dqn`, where training is taking place.
- Added other codes related to the questions.

Additional implementation for sub questions and result analysis

Subquestion 3

- Explore training performance with different hyperparameters: learning rate, batch sizes, and discount factors — and report their impact. Pick three values for each hyperparameter: roughly “low”, “medium”, and “large” (use your judgment).

Implementation:

Chosen the corresponding values for `learning_rate`, batch sizes and discount factor

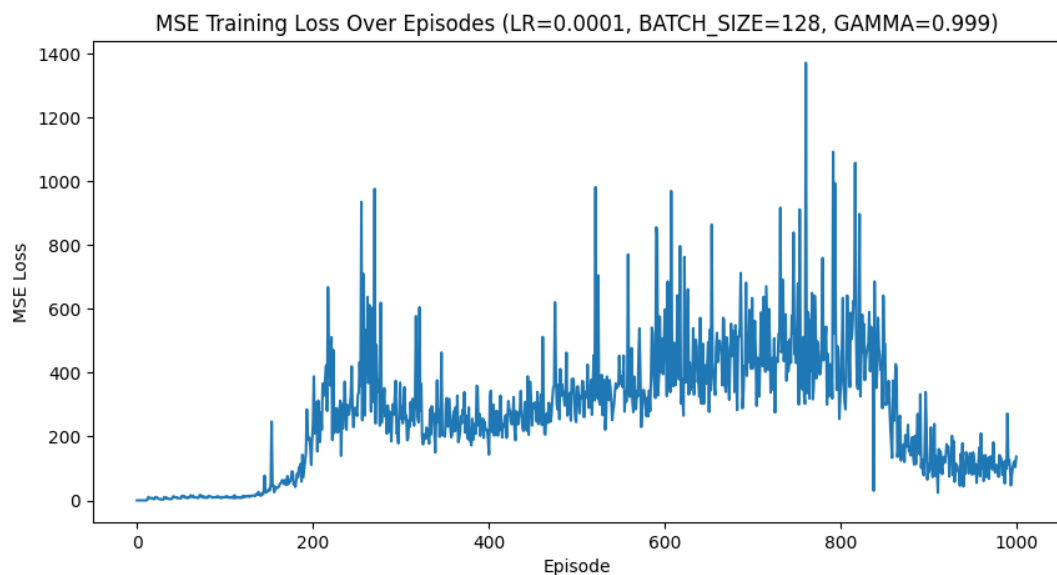
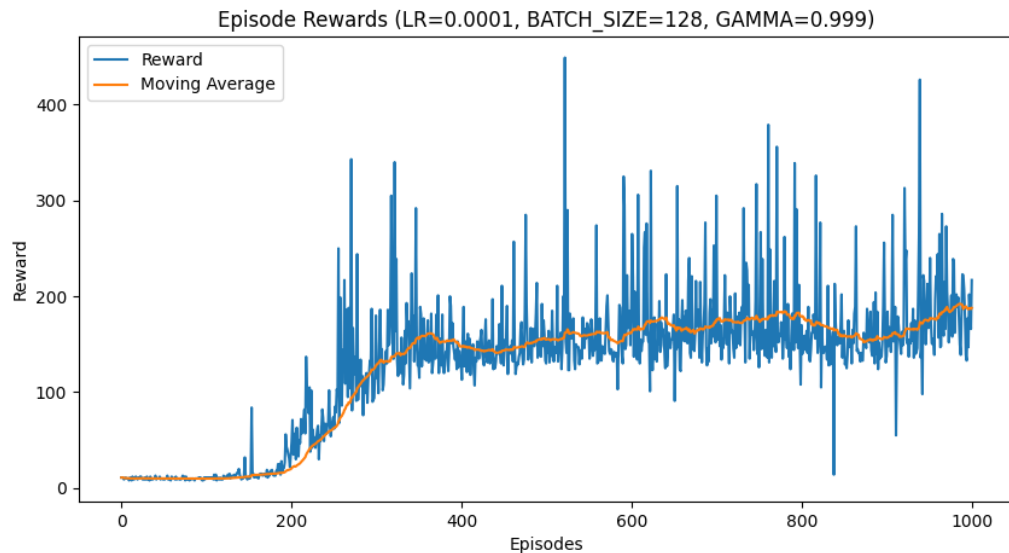
- `learning_rates = [0.0001, 0.001, 0.01]`
- `batch_sizes = [32, 64, 128]`
- `gamma_values = [0.9, 0.99, 0.999]`

Due to time constraints training is done on the 9 combinations of these chosen values.

- {'LR': 0.0001, 'BATCH_SIZE': 128, 'GAMMA': 0.999):

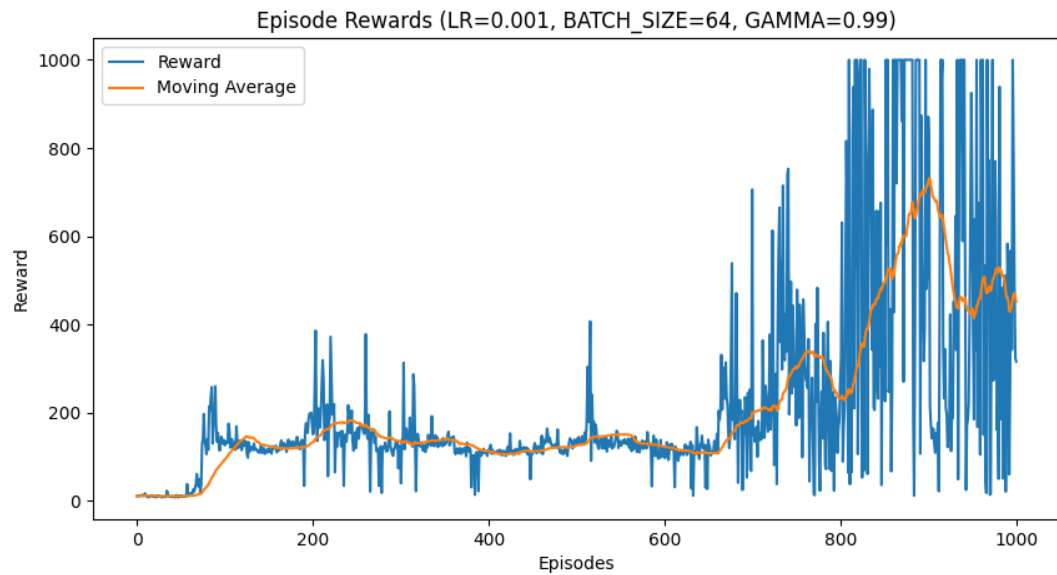
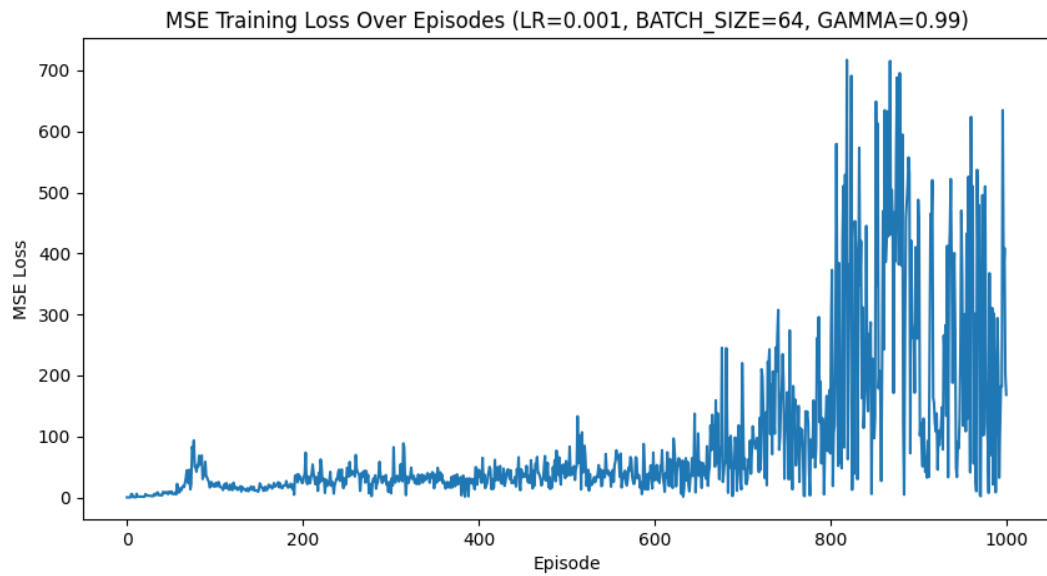
The max reward is between 400-450, learning is happening as we can see the moving average. MSE losses increase and towards the last training phase it reduces.

Learning rate : low, batch size :high, gamma: high , as learning rate is low it's initial performance in training is low. It's not suitable for best setting due to slow learning rate , it require more time to reach the maximum reward.



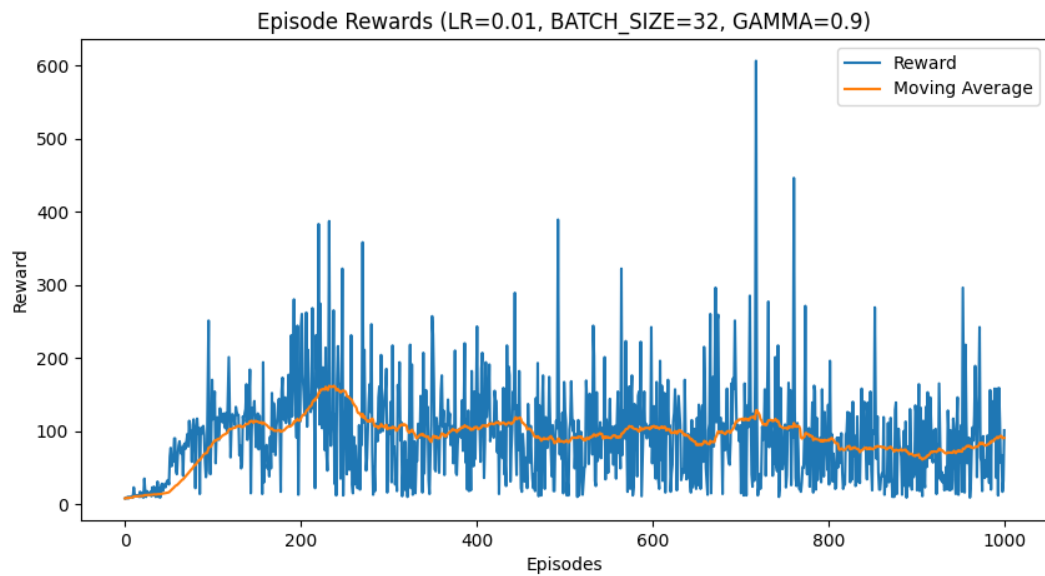
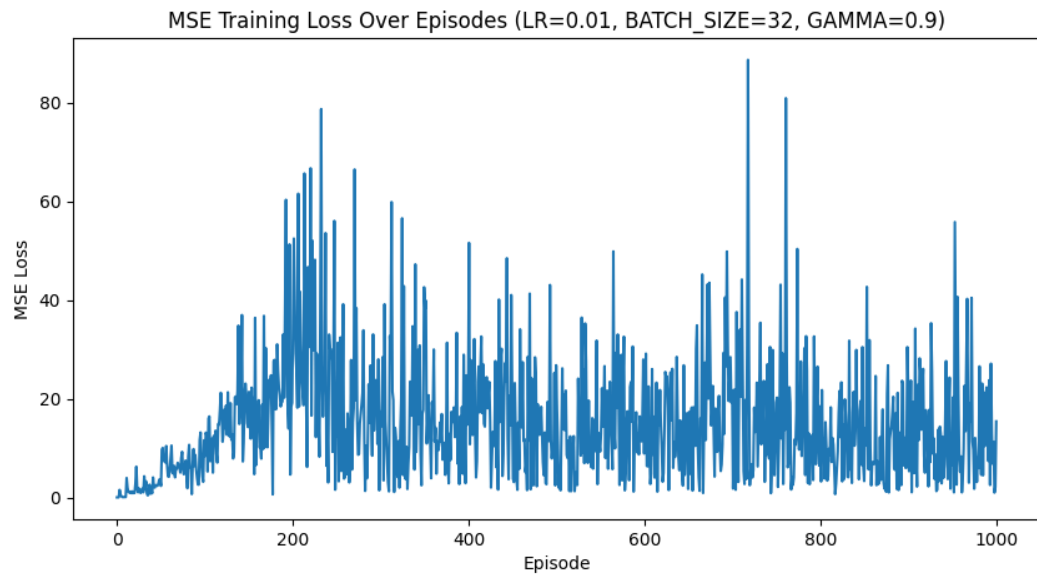
- {'LR': 0.001, 'BATCH_SIZE': 64, 'GAMMA': 0.99}

Stable performance, the learning rate , batch size and gamma are in the medium level.
Max reward is 1000, MSE loss increases at last phase.



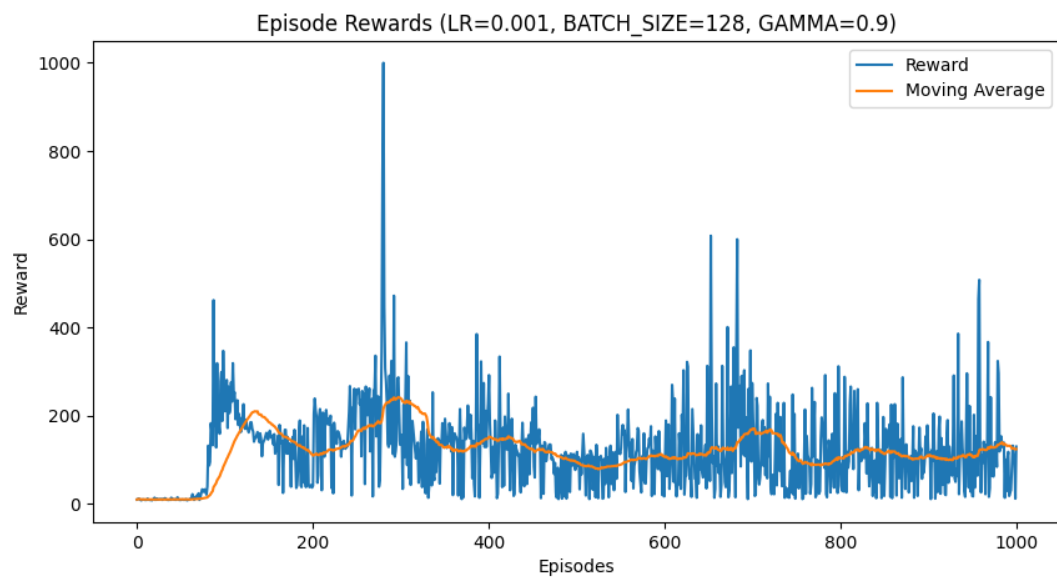
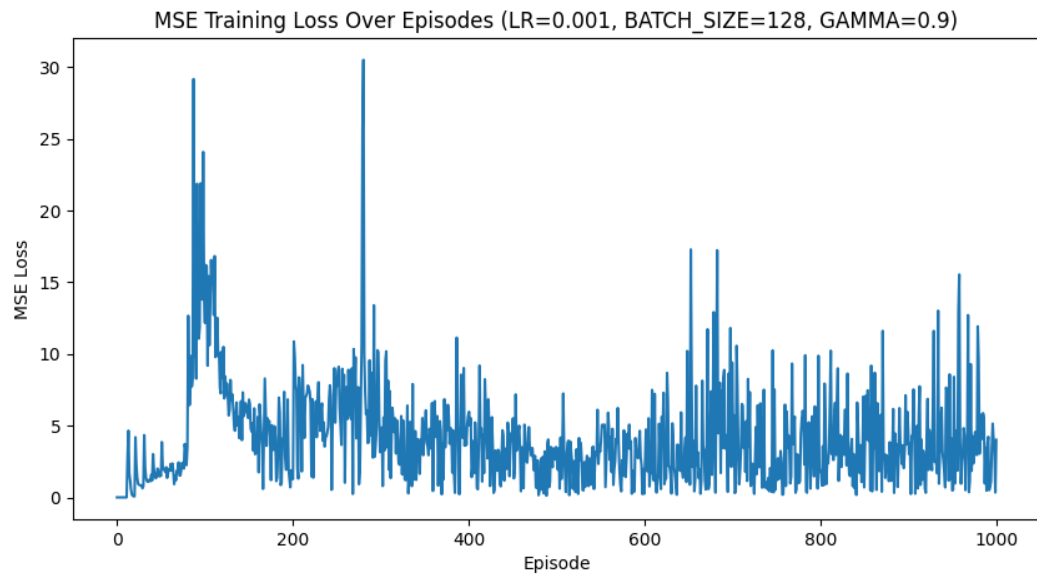
- **{'LR': 0.01, 'BATCH_SIZE': 32, 'GAMMA': 0.9}**

High learning rate and low gamma gives instability . Convergence doesn't happen. The maximum reward is almost 600. So, this is not the best setting for training convergence and performance.



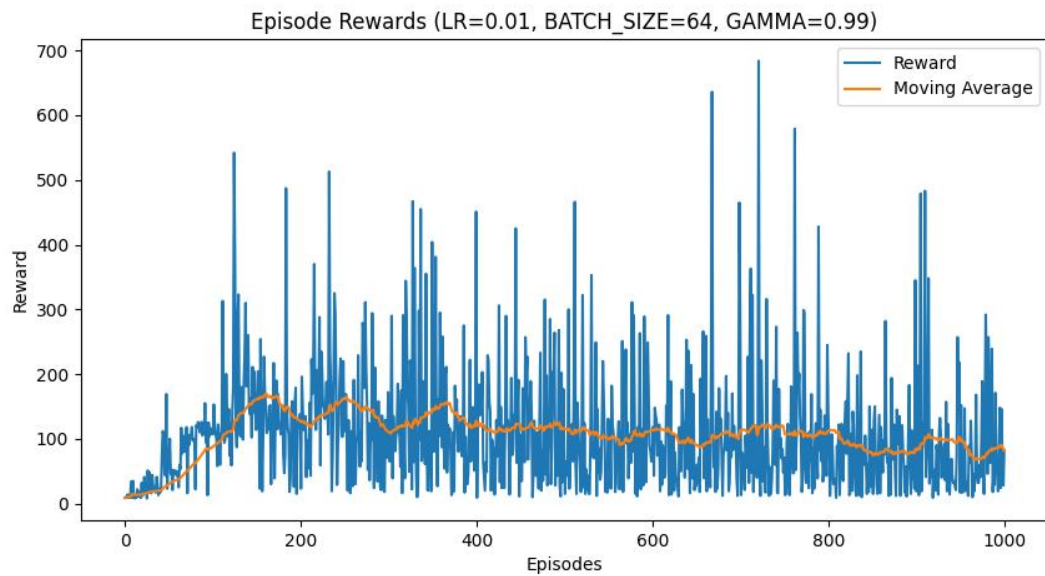
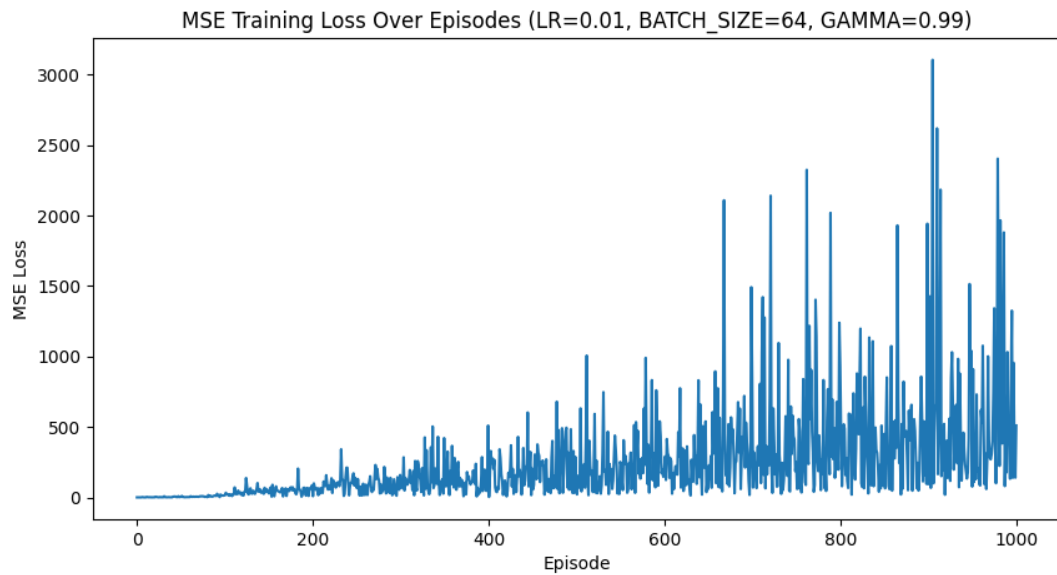
- {'LR': 0.001, 'BATCH_SIZE': 128, 'GAMMA': 0.9}

High batch size and low gamma, which indicate the training is slow and look forward only for short term rewards, no this is not a best setting.



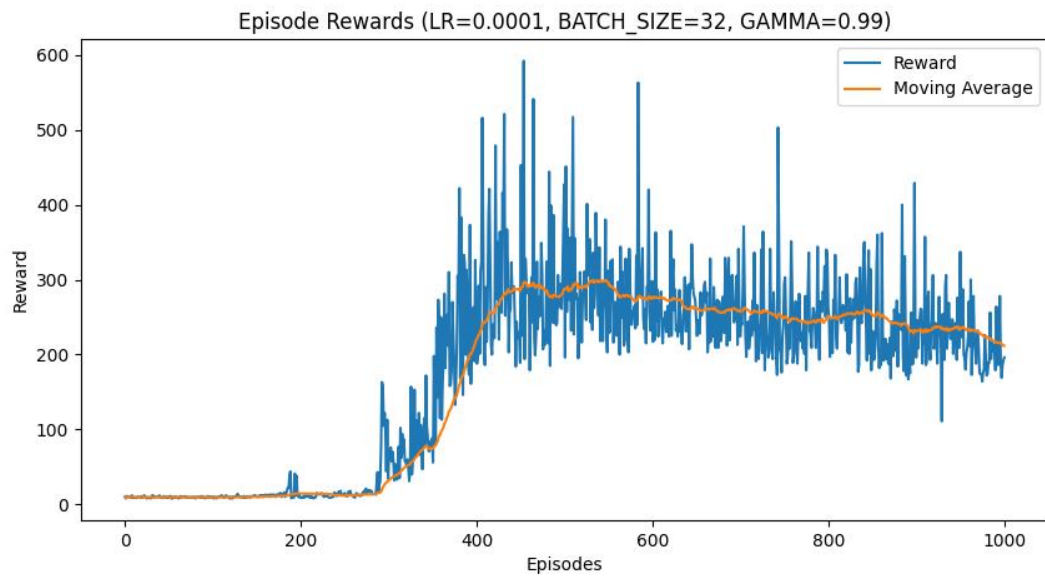
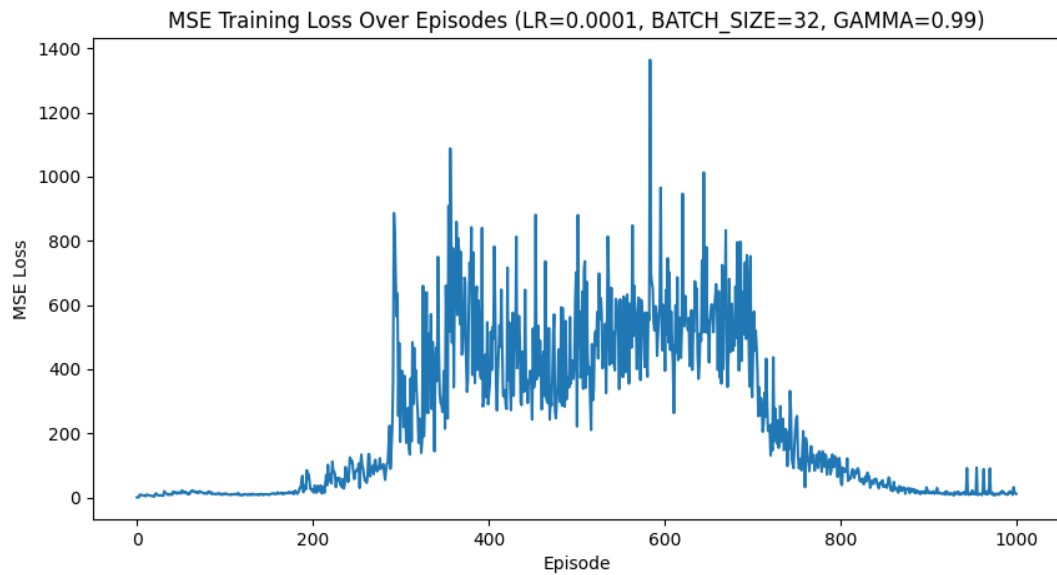
- {'LR': 0.01, 'BATCH_SIZE': 64, 'GAMMA': 0.99}

High learning rate which causes unstable performance inspite of having good gamma. Here the moving average is oscillating and not converging so this is not a good setting for the training.



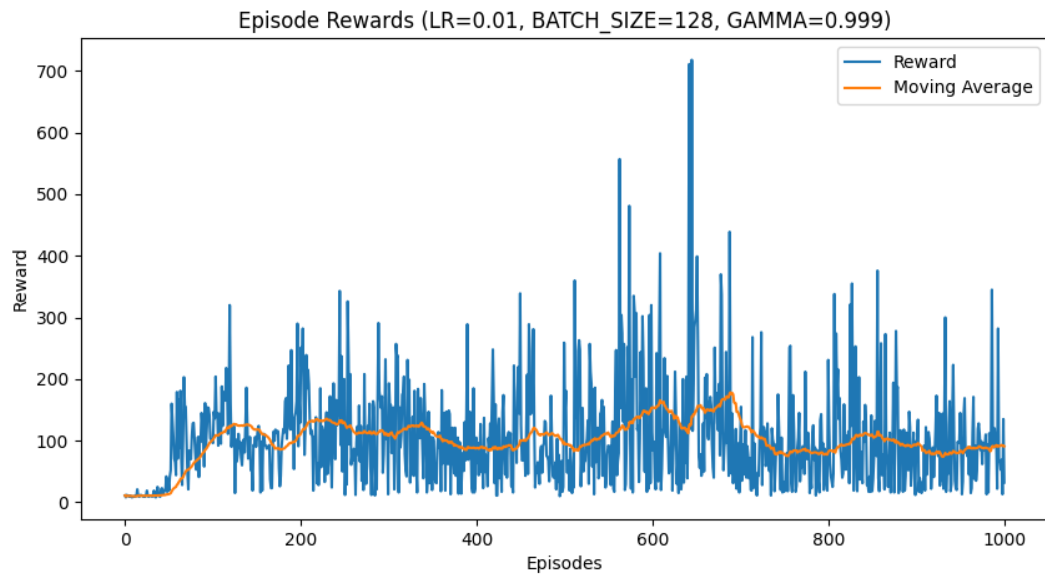
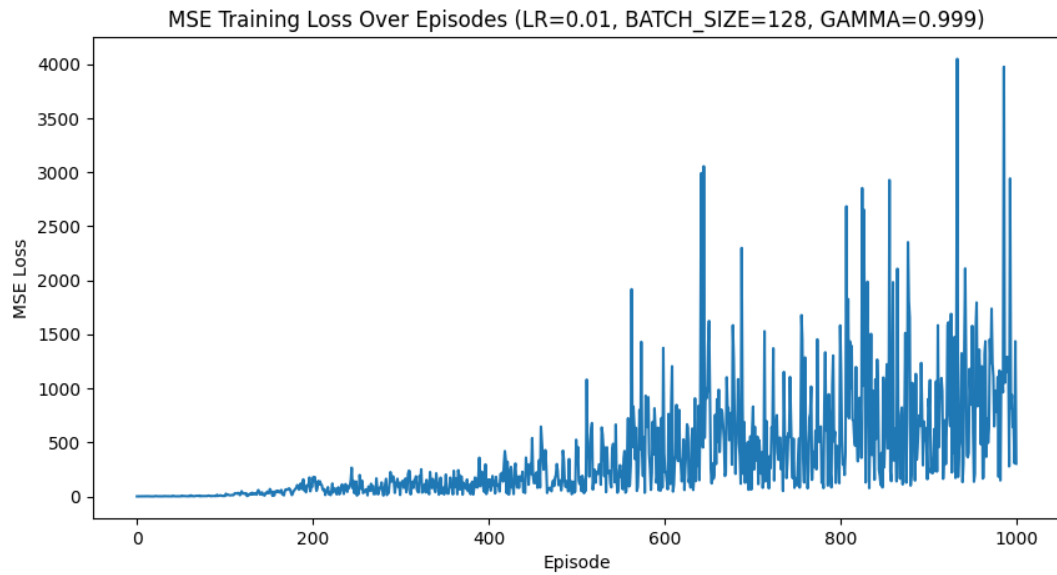
- {'LR': 0.0001, 'BATCH_SIZE': 32, 'GAMMA': 0.99}

Small learning rate and batch size, it takes long time to learn. MSE loss and the moving average converges but it still requires time to learn, so I am not choosing it for our 1000 episode training.



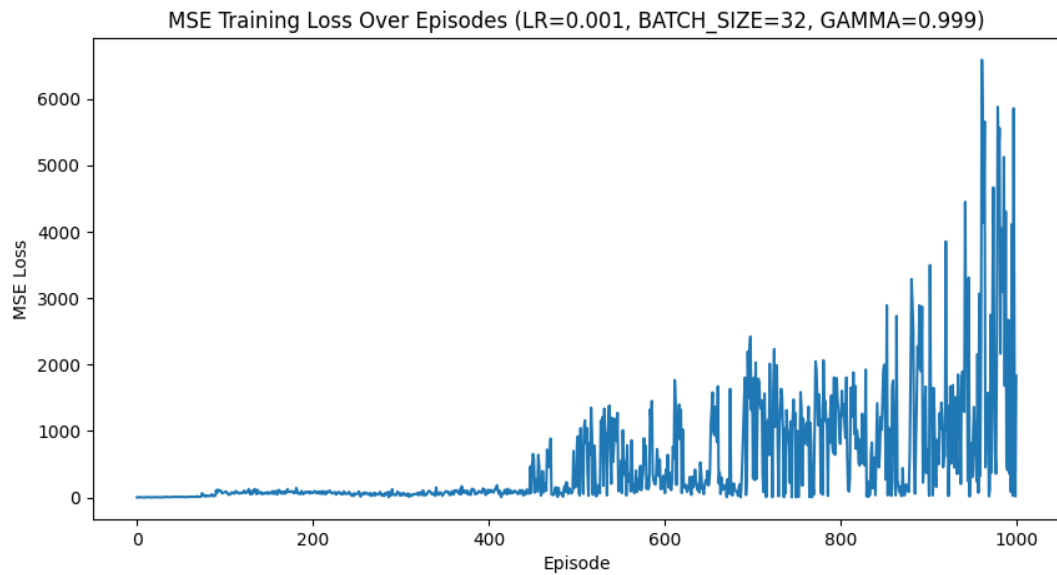
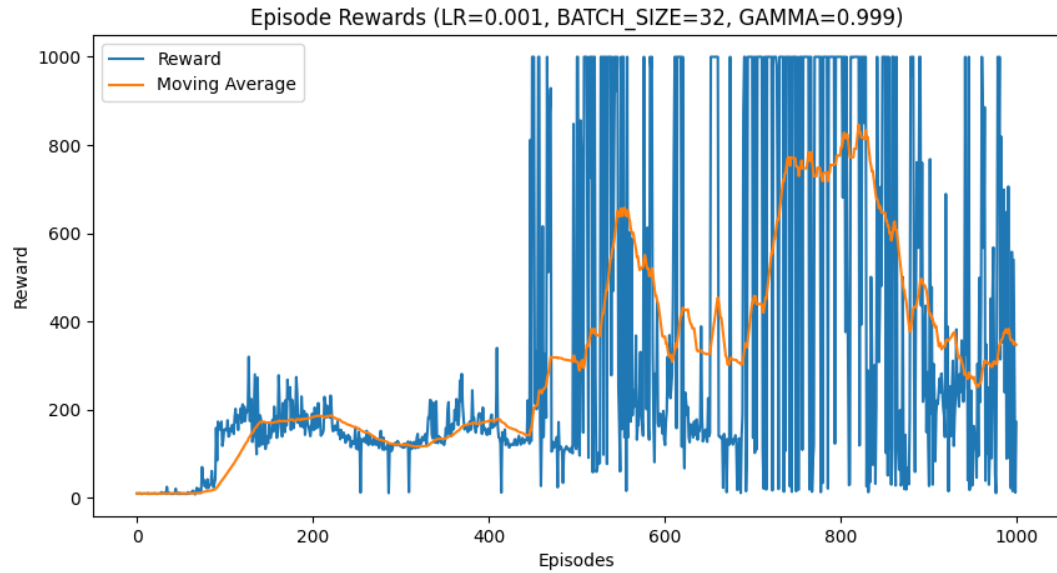
- **{'LR': 0.01, 'BATCH_SIZE': 128, 'GAMMA': 0.999}**

High learning rates, high batch size and high gamma, this causes aggressive updates and not converging , so It's not a good settings for the training at the given other parameters



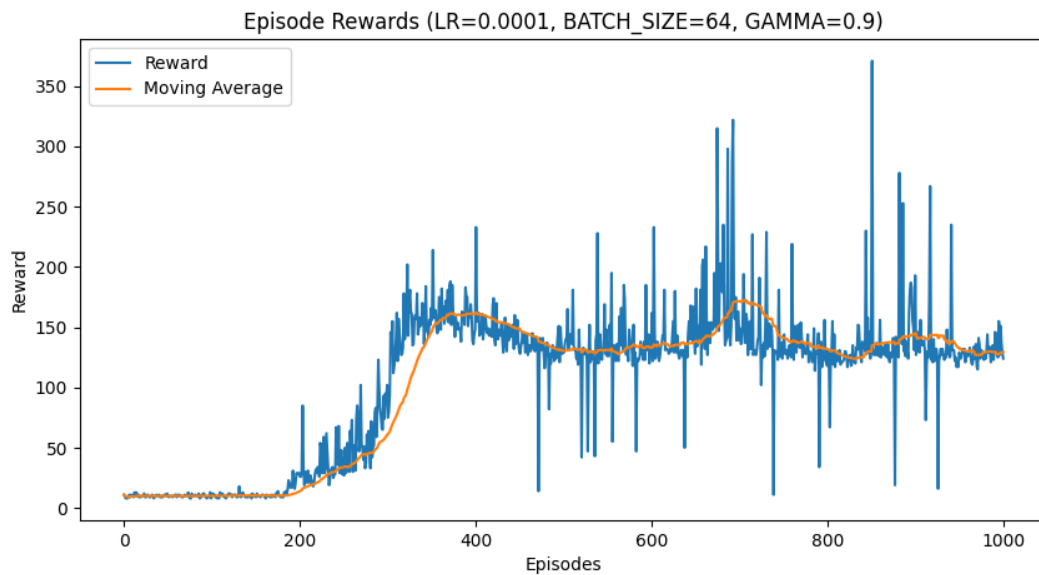
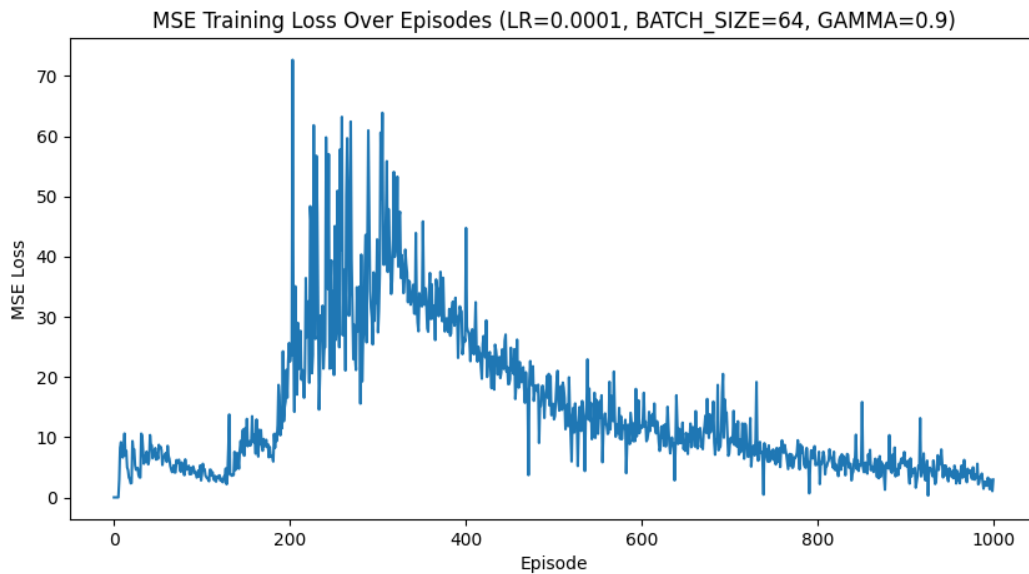
- {'LR': 0.001, 'BATCH_SIZE': 32, 'GAMMA': 0.999}

Medium leaning rate , low batch size and high gamma, it causes gradual increase in MSE loss but there is noise in the moving average. It's a decent one but require some tuning.



- {'LR': 0.0001, 'BATCH_SIZE': 64, 'GAMMA': 0.9}

Learning rate is low, batch size is medium, and gamma is low . Here the reward is almost 350 with a flat trend. Not suitable for best setting as it is focused on short term goals and slow learning



Conclusion:

Learning Rate	Batch Size	Gamma	Avg Reward (Last 100)	Max Reward
0.001	64	0.99	437.44	1000
0.01	32	0.9	81.25	606
0.001	128	0.9	116.96	1000
0.01	64	0.99	90.5	684
0.0001	32	0.99	222.34	592
0.01	128	0.999	86.58	718
0.001	32	0.999	313.58	1000
0.0001	64	0.9	130.62	371

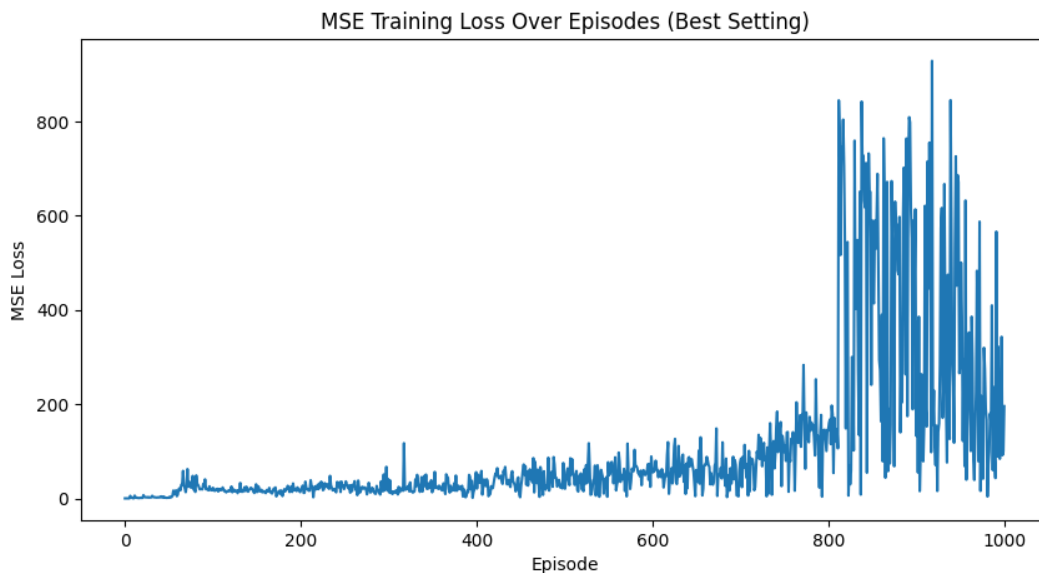
Hyperparameter	low	medium	high
Learning rate	Will be stable but the coverage will be slow	balanced	Instability
Batch size	Faster updating but noisy	balanced	Slower updates but stable
Gamma	Only care about short term rewards	Balanced	Care about future rewards, slow and unstable learning

From the above run 9 combination of different learning rate, gamma and batch size, I am choosing the combination of (0.001, 64, 0.99) as the best setting.

Subquestion 1

Plot the MSE training loss (of the NN) over episodes to demonstrate convergence. For your best setting.

Answer: The MSE training loss over episodes is very low during initial training episodes and it increases and reaches a convergence state at the end of the episode. No extreme spikes or sudden increases this shows numerical stability. Smooth decline or stabilization of the loss, indicating the neural network's Q-value predictions are becoming more accurate.

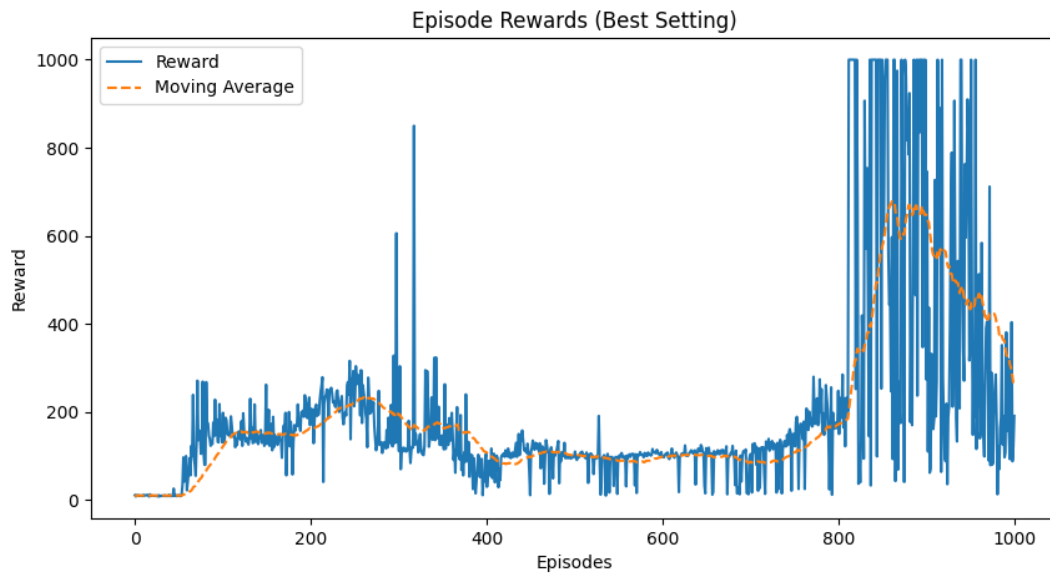


Subquestion 2

Plot the raw reward and moving average of the reward (over a window of 50 episodes) to show the convergence

Answer:

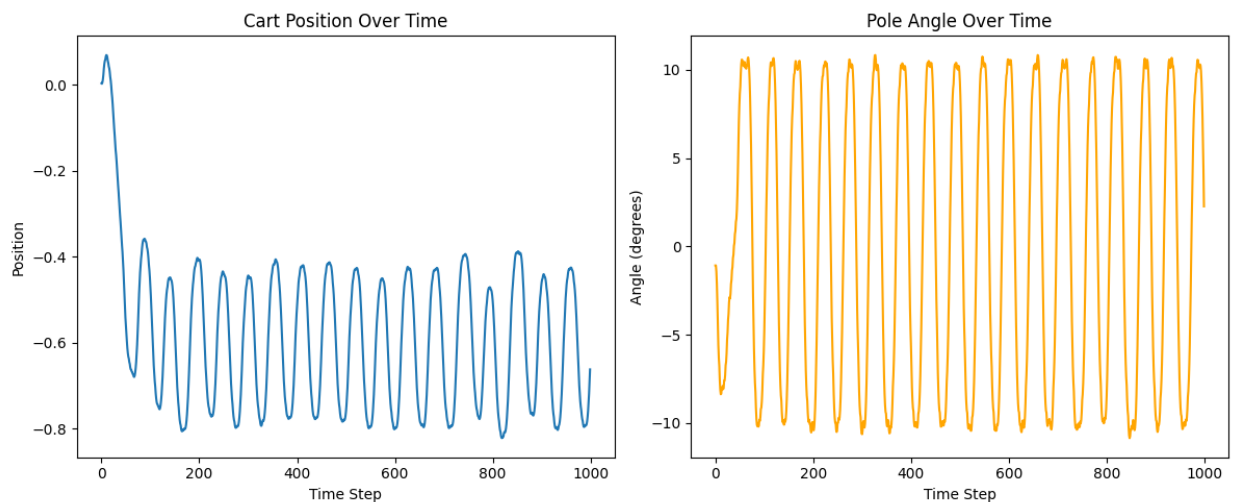
Raw reward is noisy per episode due to ϵ -greedy exploration. But the values consistently improve, especially after 800 episodes. Some dips are natural during exploration. Shows a smooth, steep upward trend, reaching near 1000. Stays flat and high in the final 800–1000 episodes show clear convergence. The moving average shows a clear improvement in performance, with fewer fluctuations.



Subgestion 4

Test your final trained NN with $\epsilon = 0$. Plot the position and angle over time

Answer: Position of the cartpole starts at 0 and reach fluctuating between -0.4 and -0.8 values. Here cartpole is always balancing 0–1000-time steps, which is a good indicator of that agent properly learns the model. The pole angle is oscillating between -10 to 10 and covers all time steps to 1000 without terminating . The oscillations are consistent, demonstrating the agent's ability to control the pole and prevent it from falling.

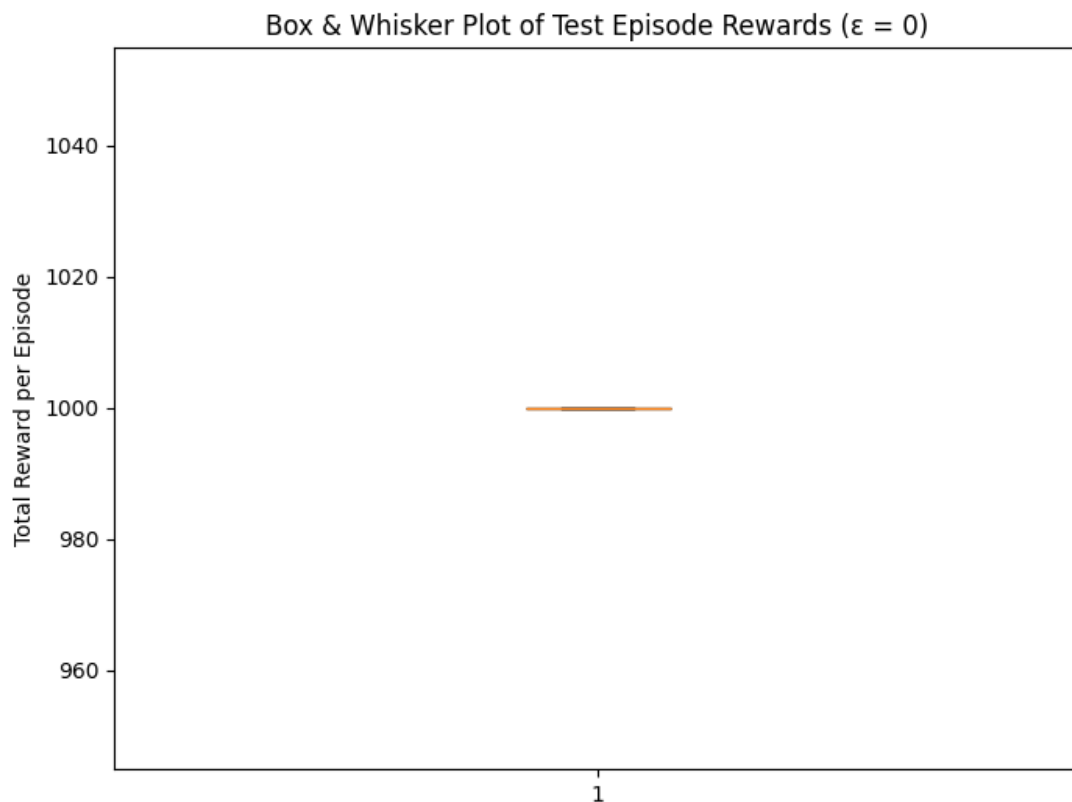


Subquestion 5

Test your final trained NN with $\epsilon = 0$ over 100 episodes and report the box & whisker plot of its rewards. The distribution of initial states should be sampled by calling `env.reset()`.

Answer:

The box and whisker plot shows a single line at a reward value of approximately 1000. This demonstrates the agent's reliability and robustness in solving the CartPole task after resetting the environment, but at the same time must check if the model is overfitting. Confirmed the env resetting is implemented in code.



Resources for reference:

- Open AI gym documentation
- GitHub repo: https://github.com/johnnycode8/gym_solutions
- Internet: for solving installation dependency issues, programming issues, and some support over graphs.